

IN ACADEMIC PARTNERSHIP WITH



Exploring Sales Determinants and Predicting Revenue in Retail Businesses

Author: Rishu Prajapati

Student ID: 26140515

Date: 06/08/2026

Wordcount: 2010

Confidential: Yes

**Faculty of Computing, Engineering and the Built
Environment**

Birmingham City University (BCU)

Sunway College, Kathmandu

Table of Contents

Abstract	1
1. Domain Description	2
2. Problem Definition.....	2
3. Literature Review.....	2
3.1. Machine Learning Methodologies	2
3.2. Key Findings and Performance Analysis	3
4. Dataset Description	3
4.1. Overview	3
4.2. Column Description	4
5. Data Preparation.....	5
5.1. Importing Libraries and Loading Dataset	5
6. Data Cleaning.....	5
6.1. Missing Value Detection	5
6.2. Duplicate Row Check	6
6.3. Feature Engineering.....	6
6.4. Dropping Irrelevant Columns	6
6.5. Feature and Target Definition	7
7. Descriptive Statistics.....	7
8. EDA (Exploratory Data Analysis)	8
8.1. Revenue Distribution	8
8.2. Total Revenue by Product Line.....	8
8.3. Quantity vs Revenue	9
8.4. Unit Price vs Revenue.....	10
9. Data Preprocessing.....	10
9.1. Train- Test Split.....	10
9.2. Label Encoding	11
10. Correlation Analysis.....	12
11. Outlier Analysis	13
12. Model Development.....	13
12.1. Machine Learning Model Used	13

12.2.	Model Training.....	13
12.3.	Model Evaluation.....	14
13.	Data Visualization	15
13.1.	Predicated and Annual Revenue	15
13.2.	Feature Importance	16
13.3.	Pair Plot: Quantity, Unit Price, Rating & Revenue.....	17
13.4.	KDE Plots of Numerical Features.....	18
13.5.	Overlapping KDE- All Numerical Features.....	18
14.	Conclusion	19
15.	References	20

List of Figures and Tables

Table 1: Column Description of Dataset

Table 2: Descriptive Statistics Summary

Figure 1: Dataset

Figure 2: Importing Libraries and Loading Dataset

Figure 3: Dataset Overview After Cleaning

Figure 4: Feature and Target Definition

Figure 5: Revenue Distribution

Figure 6: Total Revenue by Product Line

Figure 7: Quantity vs Revenue

Figure 8: Unit Price vs Revenue

Figure 9: Creating Train-Test Split

Figure 10: Label Encoding

Figure 11: Correlation Heatmap

Figure 12: Outlier Analysis

Figure 13: Model Training Configuration

Figure 14: Model Evaluation Code

Figure 15: Predicted vs Actual Revenue

Figure 16: Feature Importance

Figure 17: Pair Plot

Abstract

The retail sector has become fiercely competitive and gut feel decision-making is steadily being replaced with data-driven models. In this work we explore how sophisticated retail analytics and machine learning can be employed to increase the efficiency of supermarket operations and how we can effectively improve business forecasting. In this context, we performed analyses on the characteristics of transaction patterns, product sales performance and key revenue drivers using supermarket sales data for one comprehensive study. This empirical data is contextualized within and discussed with the standard predictive modeling framework: statistical learning; using decision tree based Random Forest models. The outcomes are then analyzed, and it's demonstrated how they provide practical and actionable business intelligence for retailers for controlling inventories, increasing customer satisfaction and gaining competitive advantage.

Key words: Retail Analytics, Supermarket Sales, Machine Learning, Data-Driven Decisions, Predictive Modeling, Random Forest, Decision Tree

1. Domain Description

The domain selected for this study is Business Sales Analysis in the retail sector, with a focus on exploring the factors that influence sales performance and predicting revenue. The reason behind choosing this domain was the retail businesses generate vast amounts of transactional data that provide valuable insights into customer behavior, purchasing patterns, product performance and revenue generation (Ravi & Raman, 2018) along with the increase of data-driven approaches taken by retail industry to improve decision-making, optimize inventory management, enhance customer satisfaction and maximize profit (Xu et al., 2020). Understanding the factors that affects the sales and predicting the revenue helps the businesses forecast the future demands, minimize losses and enhance customer satisfaction. Applying predictive analytics to this sector does not just provide a theoretical exercise; it delivers highly practical, actionable strategies that empower retailers to make smarter inventory choices, mitigate operational risks and build a resilient foundation for long-term business growth.

2. Problem Definition

As stated previously, retail businesses are currently overwhelmed by data, which they are continuously creating in massive amounts through transactions that yield numerous types of data information valuable for inventory optimization, customer satisfaction, and maximization of profit. Many businesses are facing difficulty with managing and organizing this data to determine which factors are driving the sales revenue and therefore can be used for data driven decision-making. This is because without an organized system for discerning which elements are actually responsible for driving the sales revenue the management teams must use their own intuition instead of making data-driven decision-making decisions (Brynjolfsson et al., 2011). This inefficiency places them at risk of survival, and therefore they can end up in an avalanche of debt and loan, which can lead to their eventual permanent closing.

3. Literature Review

3.1. Machine Learning Methodologies

Retail sales forecasting has rapidly evolved into one of the most critical and impactful applications of machine learning within the domain of business and commercial sector, offering to be a powerful tool to transform chaotic market variables into predictable trends for optimized inventory and maximize profits. Historically, businesses relied heavily on rigid statistical modes to guess the future demand, but a comparative study by Mustapha and Sithole (2025) demonstrated that modern machine learning models consistently outperformed traditional time-series techniques by successfully capturing complex, non-linear sales pattern and subtle hidden dependencies. In addition to this, the empirical work done by Ogunseye et.al (2024) evaluated various algorithm including Linear regression, Random Forest, Support Vector Regression and Gradient Boosting

which concluded that ensemble methods predict the sales revenue better than classical alternatives. Also, the research done by Goel et al.(2025), emphasizes that modern, data-driven algorithms excel at absorbing high-dimensional retail datasets, mitigating the risk of human error and jumping into competitive market.

3.2. Key Findings and Performance Analysis

The collective finding across recent literatures demonstrates that ensemble and tree-based machine learning provides exceptional predictive power and structural stability on datasets containing both numerical and categorical variables, making them suitable for retail transaction analysis (Janes et al., 2021). As per the calculation done by Ogunseye et.al (2024) the Random Forest algorithm achieved an impressive R^2 score of approximately 0.86 though Gradient Boosting performed slightly better. Not to mention, among all these tree-based machine learning models, specially Random Forest has been widely adopted because it reduces overfitting through bootstrap aggregation and improves prediction stability by combining multiple random trees. These findings suggest Random Forest is not only effective for revenue prediction but also valuable for identifying the relative importance of different sales determinants.

4. Dataset Description

4.1. Overview

The dataset to determine variables affecting sales and predicting sales revenue is downloaded from a Kaggle “<https://www.kaggle.com/datasets/faresashraf1001/supermarket-sales>” and the dataset contains 17 columns and 1000 rows. This dataset captures detailed transaction records from a supermarket chain in Myanmar across three cities: Yangon, Naypyitaw, and Mandalay. It provides a comprehensive view of sales activities, customer demographics, and payment methods from January to March 2019.

	Invoice ID	Branch	City	Customer	Gender	Product line	Unit price	Quantity	Tax 5%	Sales	Date	Time	Payment	cogs	gross marg	gross incot	Rating
1	750-67-84	Alex	Yangon	Member	Female	Health and	74.69	7	26.1415	548.9715	1/5/2019	#####	Ewallet	522.83	4.761905	26.1415	9.1
2	226-31-30	Giza	Naypyitaw	Normal	Female	Electronic	15.28	5	3.82	80.22	3/8/2019	#####	Cash	76.4	4.761905	3.82	9.6
3	631-41-31	Alex	Yangon	Normal	Female	Home and	46.33	7	16.2155	340.5255	3/3/2019	#####	Credit card	324.31	4.761905	16.2155	7.4
4	123-19-11	Alex	Yangon	Member	Female	Health and	58.22	8	23.288	489.048	#####	#####	Ewallet	465.76	4.761905	23.288	8.4
5	373-73-79	Alex	Yangon	Member	Female	Sports and	86.31	7	30.2085	634.3785	2/8/2019	#####	Ewallet	604.17	4.761905	30.2085	5.3

Figure 1: Dataset

4.2. Column Description

Columns	Data Type	Description
Invoice ID	String	A unique identifier for each transaction.
Branch	String (Categorical)	Branch code: A (Yangon), B (Mandalay), C (Naypyitaw)
City	String (Categorical)	City corresponding to the branch location
Customer type	String (Categorical)	"Member" or "Normal" customer
Gender	String (Categorical)	Customer gender: Male or Female
Product line	String (Categorical)	Product category (6 types)
Unit price	Float	Price per unit of the product sold (USD)
Quantity	Integer	Number of units purchased in the transaction
Tax 5%	Float	5% tax applied on the cost of goods sold (COGS)
Total	Float	Total invoice amount including tax (USD)
Date	String/DateTime	Date of the transaction (MM/DD/YYYY format)
Time	String	Time of the transaction (HH:MM format)
Payment	String (Categorical)	Payment method: Ewallet, Cash, or Credit card
COGS	Float	Cost of goods sold (Unit price x Quantity)
Gross Margin Percentage	Float	Gross margin percentage (constant at 4.761905%)
Gross Income	Float	Profit from each transaction
Rating	Float	Customer satisfaction rating on a scale of 1 to 10
Invoice ID	String	A unique identifier for each transaction.
Branch	String (Categorical)	Branch code: A (Yangon), B (Mandalay), C (Naypyitaw)

Table 1: Column Description of Dataset

5. Data Preparation

5.1. Importing Libraries and Loading Dataset

The dataset from Kaggle was loaded into Google Collab and the libraries required for the develop and evaluate Random Forest Regression model.

```
# Import all required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

# Plot styling
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 5)
plt.rcParams['font.size'] = 12

# Load dataset
df_raw = pd.read_csv('supermarket_sales.csv')
print(f"Dataset loaded successfully!")
print(f"Number of Rows    : {df_raw.shape[0]}")
print(f"Number of Columns: {df_raw.shape[1]}")
print(f"\nColumn Names:")
for col in df_raw.columns:
    print(f"    - {col}")
```

Figure 2: Importing Libraries and Loading Dataset

6. Data Cleaning

6.1. Missing Value Detection

Missing value detection was performed for the whole dataset under the variable miss_val and the analysis resulted in zero missing values.

6.2. Duplicate Row Check

The duplicate row check was done where none of the rows were found to be duplicate resulting in 1000 rows and 17 columns.

6.3. Feature Engineering

In order to capture time-based buying behavior, the date and time data had to be parsed into individual readable elements. This script breaks down the date-time strings into their month, day of the week, and precise time of day, providing readable numerical features rather than incomprehensible data strings. This allows the model to quickly identify meaningful trends in shopping behaviors such as busy shopping weekends, seasonal buying patterns, or even specific busy times of day.

6.4. Dropping Irrelevant Columns

Irrelevant columns were removed from the dataset to improve the model performance and prevent data leakage. After removing those irrelevant columns only 13 columns and 1000 rows were left.

Dataset Overview After Cleaning

	Column	Data Type	Missing Values
0	branch	object	0
1	city	object	0
2	customer_type	object	0
3	gender	object	0
4	product_line	object	0
5	unit_price	float64	0
6	quantity	int64	0
7	payment	object	0
8	rating	float64	0
9	revenue	float64	0
10	month	int32	0
11	day_of_week	int32	0
12	hour	int32	0

Figure 3: Dataset Overview After Cleaning

6.5. Feature and Target Definition

The target variable (y) is defined as transaction revenue (Revenue = Unit Price x Quantity). The input features (X) consist of all the independent variables that are remained after removing dropping irrelevant columns.

```
# Add Revenue target
# (Revenue = Unit price x Quantity, computed before split - no leakage)
df_full['revenue'] = df_full['unit_price'] * df_full['quantity']
```

Figure 4: Feature and Target Definition

7. Descriptive Statistics

Statistic	Unit Price (\$)	Quantity	Rating	Revenue (\$)
Count	1,000	1,000	1,000	1,000
Mean	55.67	5.51	6.97	307.59
Std	26.49	2.92	1.72	181.39
Min	10.08	1.00	4.00	11.28
25%	32.88	3.00	5.50	161.85
50% (Median)	55.23	5.00	7.00	286.60
75%	77.93	8.00	8.50	456.78
Max	99.96	10.00	10.00	993.60

Table 2: Descriptive Statistics

Overall, after observing all 1,000 of the transactions, the business seems to be stable and strong, as on average customers check out with an average purchase price of \$307.59, although individual registers display transactions ranging anywhere from \$11.28 to \$993.60 (one of a kind fast impulse buy vs. A shopping spree stock-up). This vast monetary gap can be expected from the individual products: prices of products range equally on a scale from \$10-\$100 and amounts purchased by customers ranges from single digits up to a maximum of ten items per shopping trip. The customer experience score is overall a positive aspect, scoring a 6.97/10 on average.

8. EDA (Exploratory Data Analysis)

8.1. Revenue Distribution

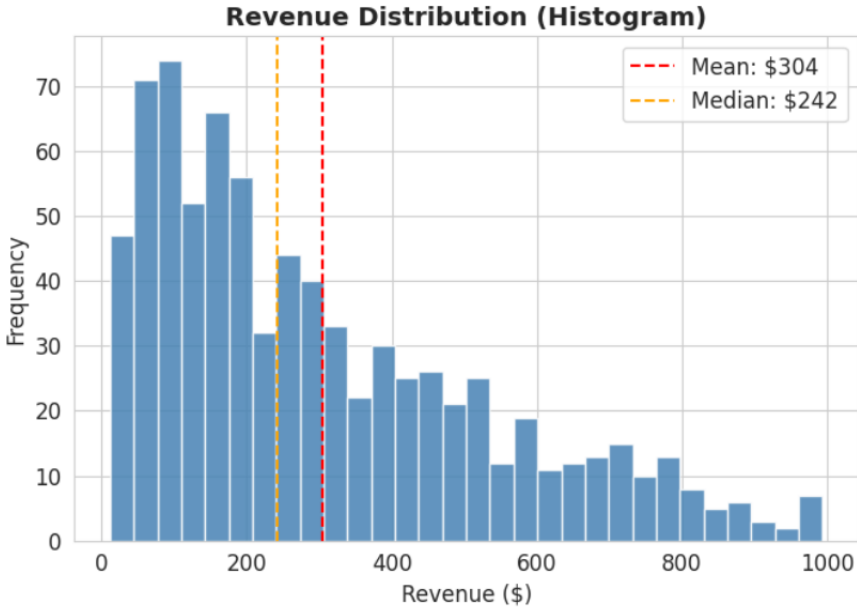
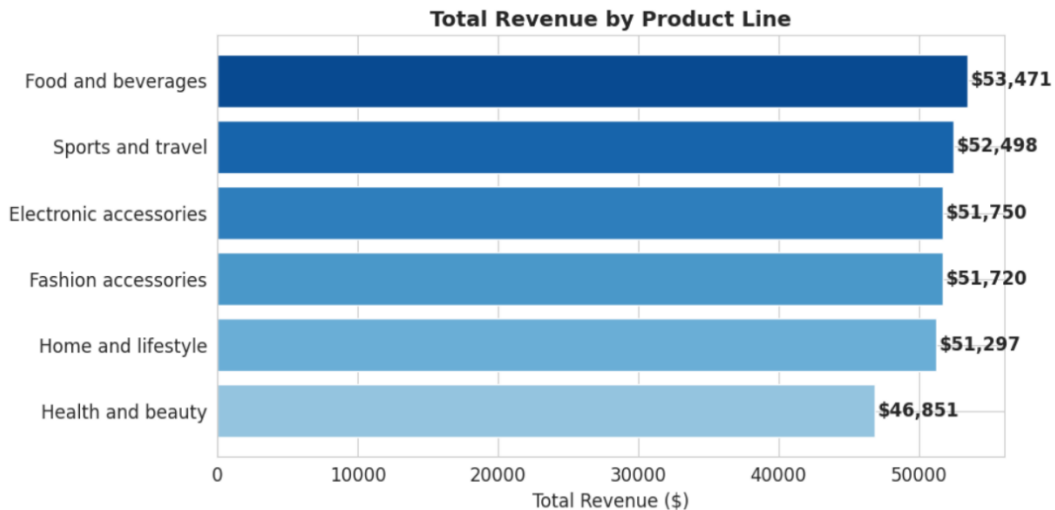


Figure 5: Revenue Distribution

The histogram shows a classic retail pattern: the revenue distribution is heavily right-skewed, meaning the vast majority of shoppers stick to smaller, lower-value baskets under \$400.

8.2. Total Revenue by Product Line



Highest revenue product line: Food and beverages (\$53,471.28)

Figure 6: Total Revenue by Product Line

Food and beverages leads the pack, bringing in \$53,471, but Sports and travel is breathing right down its neck at \$52,498. The only real outlier here is Health and beauty, which lags behind the rest at \$46,851.

8.3. Quantity vs Revenue

...

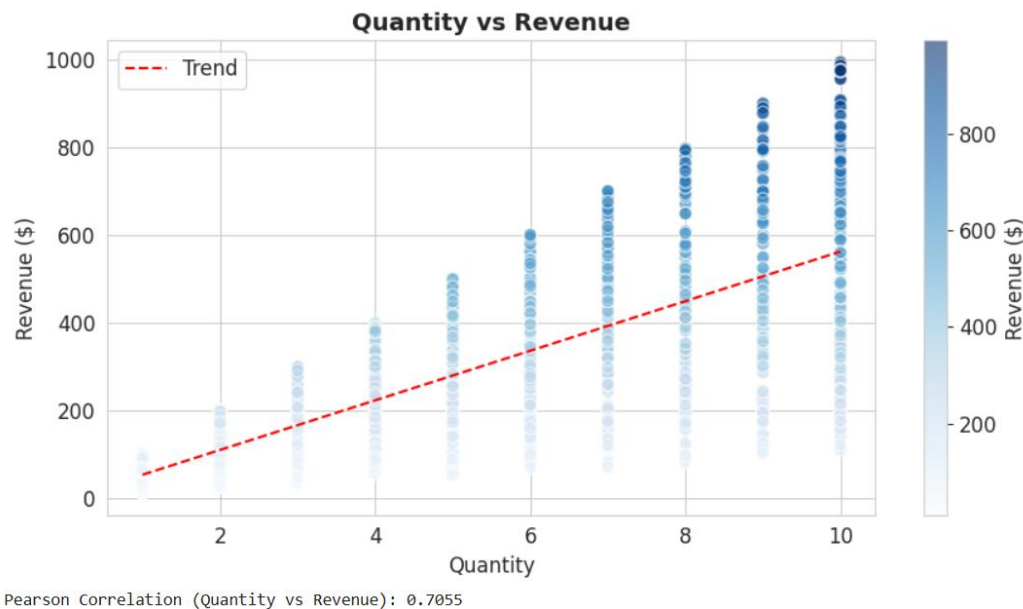


Figure 7: Quantity vs Revenue Scatter Plot

This scatter plot shows a clear, common-sense rule: the more items people buy, the more they spend, which explains the strong 0.7055 correlation. It shows that getting customers to buy in bulk is where the real magic happens, especially when they start mixing higher-priced items into those larger hauls.

8.4. Unit Price vs Revenue

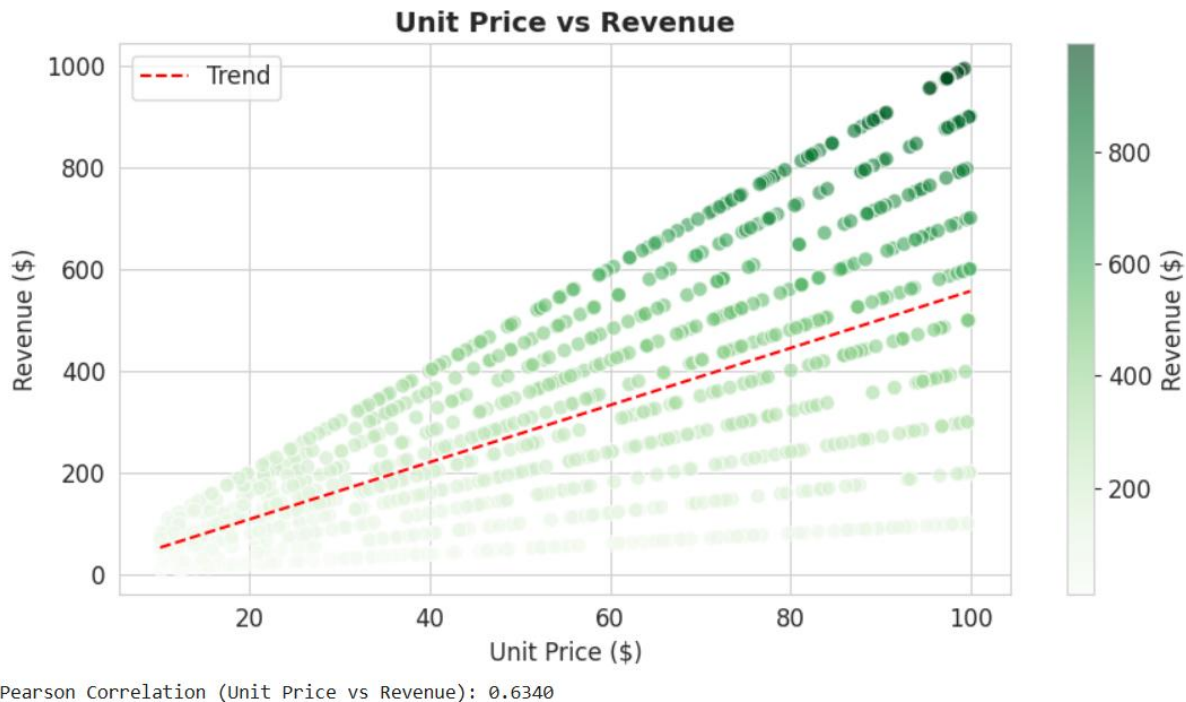


Figure 8: Unit Price vs Revenue Scatter Plot

So, a strong, positive 0.6340 correlation is visible here, the total bill simply goes up as items cost more, which is what we'd expect. The cool aspect of this chart is the striped "fan" pattern. Every diagonal line you see stands for a different number of items checked out. As you go from cheap, bulk checkout quantities to extremely high \$100 items, our revenue starts to fan out from a minimum of \$100 (1 expensive item) to \$1000 (10 expensive items). Our biggest sales come when we push high-price goods in high volumes.

9. Data Preprocessing

9.1. Train- Test Split

The dataset is split into 80% (800 rows) and 20% (200 rows) data for training and testing the model respectively using `random_state = 42`. This splitting is executed on the raw data before any preprocessing occurs to prevent data leakage.

```

df_train, df_test = train_test_split(df_full, test_size=0.2, random_state=42)

print(f"Total rows      : {len(df_full)}")
print(f"Training rows   : {len(df_train)} ({len(df_train)/len(df_full)*100:.0f}%)")
print(f"Testing rows    : {len(df_test)} ({len(df_test)/len(df_full)*100:.0f}%)")
print("\nSplit done – preprocessing will now be applied separately to each set.")

print(df_train.columns.tolist())

```

Figure 9: Creating Train-Test Split

9.2. Label Encoding

Categorical variables converted into numerical form by using Label Encoding. The encoder was applied on training data only and then also applied on both training and test sets. This ensures consistent encoding across datasets while maintaining model integrity.

```

from sklearn.preprocessing import LabelEncoder
# Standardize column names
df_train.columns = (
    df_train.columns
    .str.strip()
    .str.lower()
    .str.replace(' ', '_')
)

df_test.columns = (
    df_test.columns
    .str.strip()
    .str.lower()
    .str.replace(' ', '_')
)

# Categorical columns
categorical_cols = [
    'branch',
    'city',
    'customer_type',
    'gender',
    'product_line',
    'payment'
]

encoding_map = {}

for col in categorical_cols:
    le = LabelEncoder()

    # Fit on train and transform train
    df_train[col] = le.fit_transform(df_train[col])

    # Transform test using same encoder
    df_test[col] = le.transform(df_test[col])

    # Save mapping
    encoding_map[col] = {

```

```

        cls: int(code)
        for cls, code in zip(le.classes_, le.transform(le.classes_))
    }

print("Label Encoding Complete\n")

for col, mapping in encoding_map.items():
    print(f"{col}: {mapping}")

# — Feature Selection (done HERE, after encoding) —————
feature_cols = ['branch', 'city', 'customer_type', 'gender', 'product_line',
                'unit_price', 'quantity', 'payment', 'rating', 'month', 'day_of_week', 'hour']

X_train = df_train[feature_cols]
y_train = df_train['revenue']

X_test = df_test[feature_cols]
y_test = df_test['revenue']

print(f"\nX_train: {X_train.shape} | y_train: {y_train.shape}")
print(f"X_test : {X_test.shape} | y_test : {y_test.shape}")
print(f"\nFeatures: {feature_cols}")

```

Figure 10: Label Encoding

10. Correlation Analysis

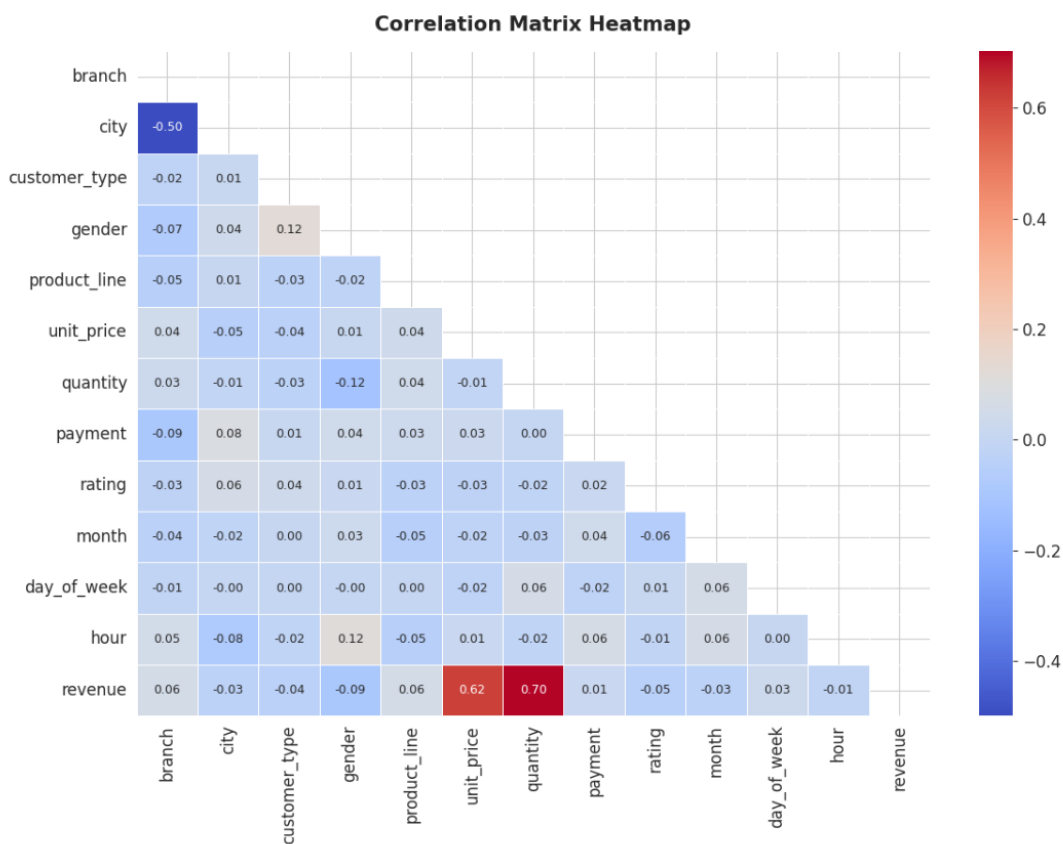


Figure 11: Correlation Heatmap

A strong red means a positive correlation and a strong blue a negative one. In the last row we only see the variables with significant impact on revenue, correlated with revenue in the positive side. Quantity and unit_price are correlated 0.70 and 0.62. Beside the perfectly explainable structural relationship between city and branch (-0.50), the rest of the matrix is an ocean of neutral colors (gray, light blue) meaning the strong independence between demographics, hour and payment mode

11. Outlier Analysis

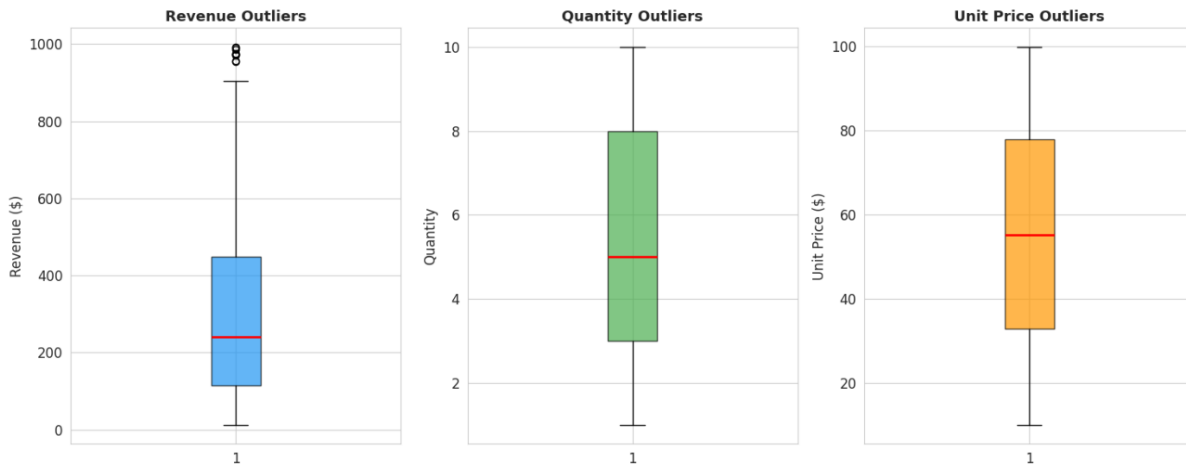


Figure 12: Outlier Analysis

The three box plots above provide a quick first glimpse of data quality—an indicator of how dispersed are our variables, and if there is any anomaly. The Quantity (green) and Unit Price (orange) appear acceptable, the Revenue (blue) does not. Those isolated dots, above the blue line, are actual outliers, the actual extreme statistical points, that is, abnormal sales transactions as far as revenue is concerned.

12. Model Development

12.1. Machine Learning Model Used

The machine learning model used for the sales revenue prediction is Random Forest. Regressor/ The Random Forest algorithm is a powerful ensemble machine learning technique that builds on the bagging (Bootstrapped Aggregation) of Decision Trees (Breiman, 2001).

12.2. Model Training

To train our predictive model, we set up a **Random Forest Regressor** with a balanced group of settings designed to keep it accurate without letting it memorize the noise in our data.

- **The Setup:** The model builds an ensemble of 200 individual decision trees, with each tree limited to a maximum depth of 10 layers so it doesn't get overly complex and overfit.
- **The Rules:** We added strict structural guardrails (`min_samples_split=4` and `min_samples_leaf=2`) that force the model to focus on broad, meaningful shopper trends rather than getting distracted by tiny anomalies.
- **The Shortcuts:** To keep things fast and smart, we used all available processor cores (`n_jobs=-1`) for rapid training, while activating the `oob_score` to act as a built-in validation check on the fly.

```
rf = RandomForestRegressor(
    n_estimators=200,
    max_depth=10,
    min_samples_split=4,
    min_samples_leaf=2,
    max_features='sqrt',
    random_state=42,
    n_jobs=-1,
    oob_score=True
)

rf.fit(X_train, y_train)
print("Model training complete!")
print(f"Number of trees: {rf.n_estimators}")
print(f"Max depth      : {rf.max_depth}")
```

Figure 13: Model Training Configuration

12.3. Model Evaluation

To see how well our Random Forest model actually performs, we ran predictions on both the training data and a separate, unseen test set. The code calculates three standard industry metrics- R^2 (accuracy score), MAE (average dollar error), and RMSE (which penalizes larger misses) and formats them into a clean, side-by-side comparison table. Printing the final Test R^2 as a percentage gives us a quick, bottom-line look at exactly how much of the store's revenue variance our model successfully figured out.

```
Out-of-Bag Score: 0.9012558333826612
Metric Train Test
R² 0.9689 0.9028
MAE $30.44 $59.15
RMSE $40.82 $75.75

Test R² = 90.28%
```

Figure 14: Model Evaluation Results

13. Data Visualization

13.1. Predicated and Annual Revenue

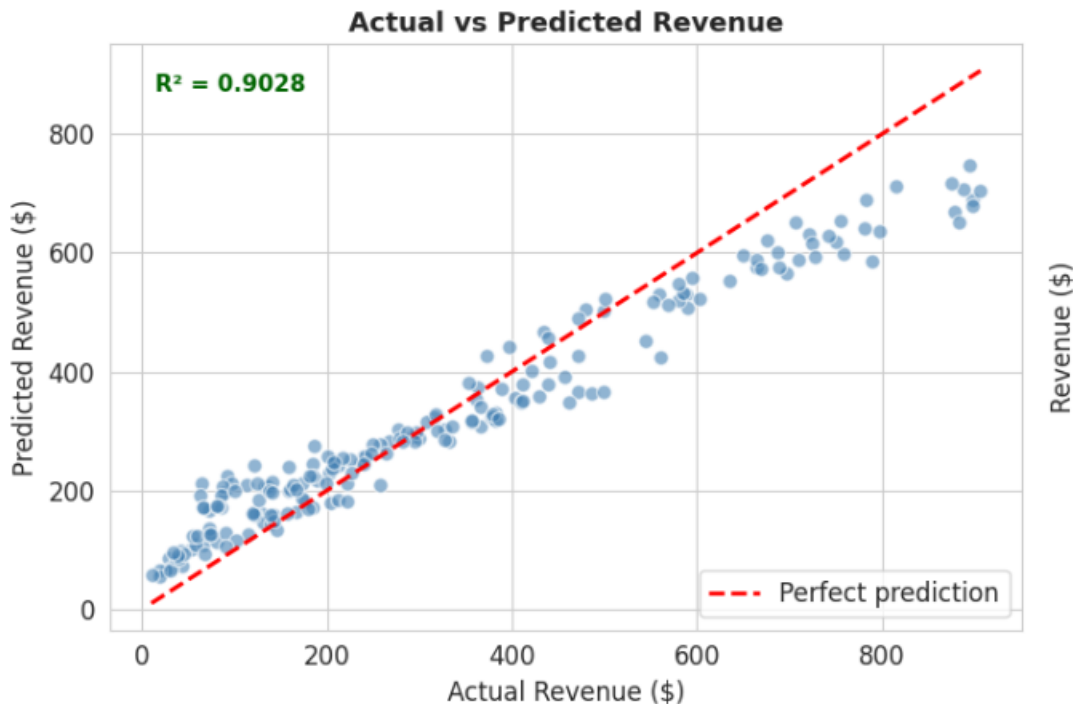


Figure 15: Predicted vs Actual Revenue

The red dashed line represents a "perfect guess"- where the predicted price perfectly matches the actual checkout bill. For the vast majority of our sales, the blue dots hug that red line beautifully, meaning our predictions are right on the money. However, if you look closely at the upper right corner, you'll notice the dots start dipping slightly below the line. This shows that when transactions get exceptionally large (upwards of \$600), the model plays it a bit safe and tends to slightly underestimate those massive, top-tier orders.

13.2. Feature Importance

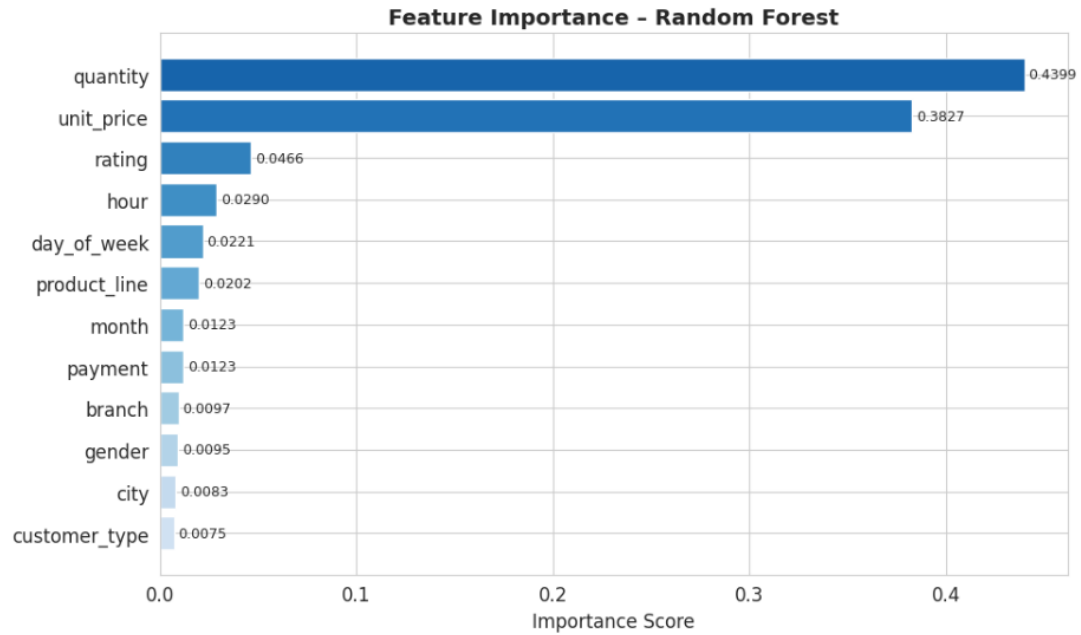


Figure 16: Feature Importance Chart

This feature importance chart reveals that our model relies almost entirely on the hard numbers of the transaction itself. Quantity (0.4399) and unit price (0.3827) absolutely dominate the graph, capturing over 82% of the decision-making power.

13.3. Pair Plot: Quantity, Unit Price, Rating & Revenue

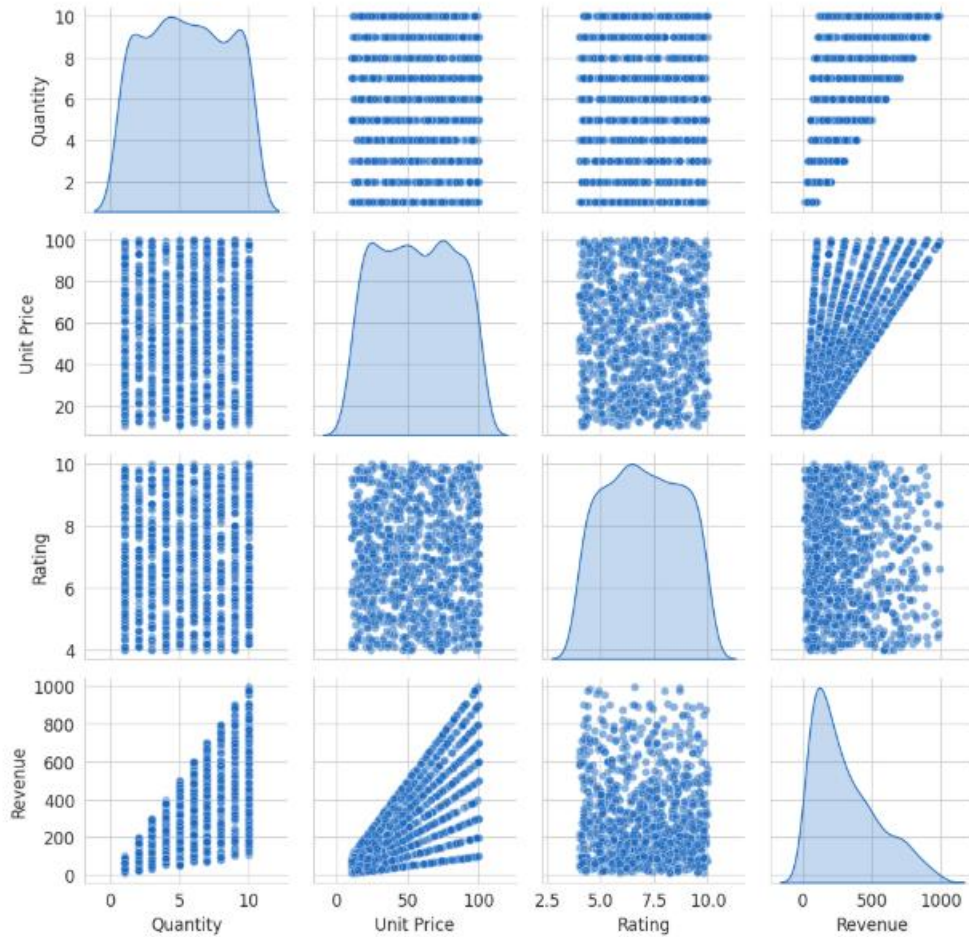


Figure 17: Pair Plot

Here we have a pair plot of our highest performing metrics. The diagonal graphs display distributions while the non-diagonal ones show relationships via scatter plot. The Rating graphs are nothing but random static, meaning customer reviews have no correlation with order quantity, price, or revenue. The Revenue graphs (last row/column) form distinct, geometrical fans. Since revenue is defined by quantity and price only, there are two neat lines of varying slope that move farther apart as quantity is increased, perfectly mirroring the mathematics involved.

13.4. KDE Plots of Numerical Features

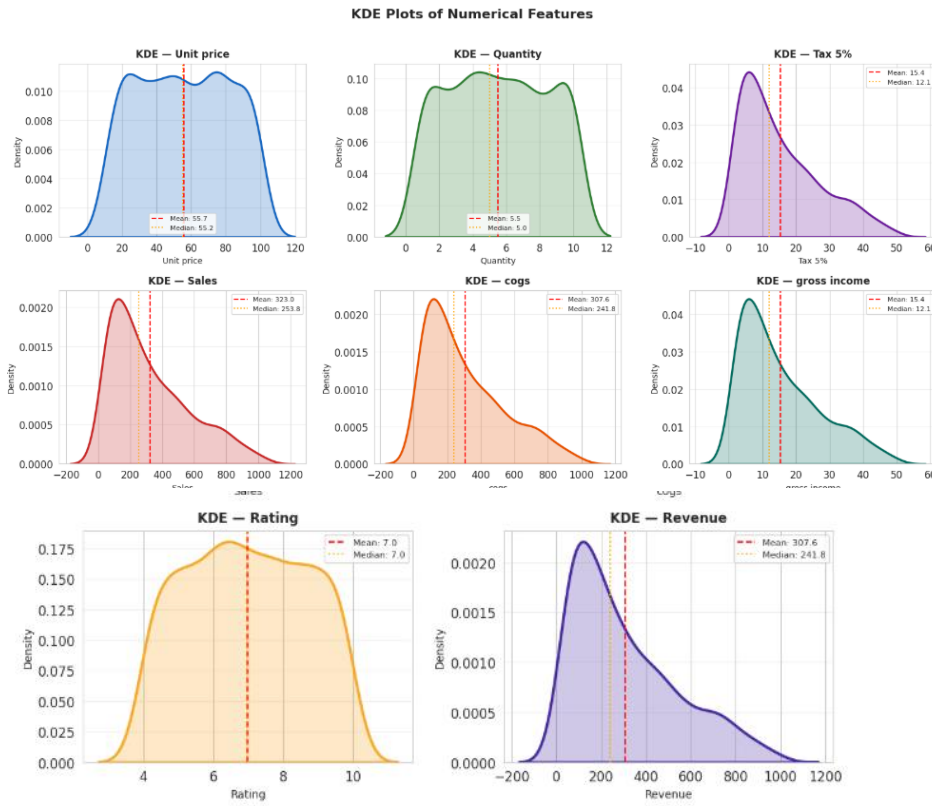


Figure 18: KDE Plots of Numerical Features

13.5. Overlapping KDE- All Numerical Features

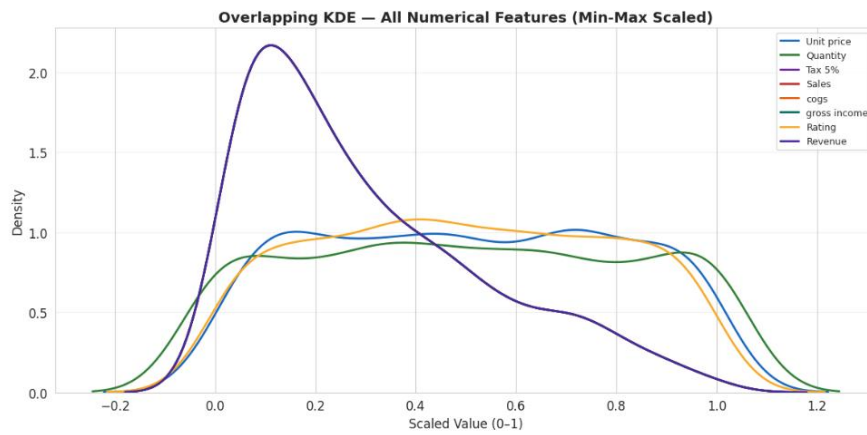


Figure 18: Overlapping KDE

14. Conclusion

Our Random Forest Regressor models 90.28% of the revenue variability on the test set. The decrease of about 6% from the training score (96.89%) is good, it indicates that there isn't an overfitting problem and we are dealing with a robust model.

The Revenue Engine: Revenue is directly and solely derived from the physical mechanism of the checkout counter. Quantity is undeniably the feature that drives everything (0.4399); it defines how the data scales up with the unit price.

Hot Operational Points: Food and beverages have been a very high revenue product line, and the high revenue city is Naypyitaw. In the checkout counter, the e-wallet has taken the first position as the preferred method of payment, whereas the female gender generates the most total revenue.

The Noise: Even if distinct demographic and operational patterns exist (e.g. Gender of the shopper or payment preferences) our visualizations show that these features do not cause transaction size increase in direct and independent way, customer profiles may bring people to store, but only their basket value and cost will determine the final amount.

15. References

- Breiman, L. (2001) Random forests. *Machine Learning*, 45 (1), pp. 5–32.
- Brynjolfsson, E., Hitt, L.M. and Kim, H.H. (2011) *Strength in numbers: how data-driven decisionmaking affects firm performance*. SSRN Electronic Journal. Available at: https://ide.mit.edu/sites/default/files/publications/2011.12_Brynjolfsson_Hitt_Kim_Strength%20in%20Numbers_302.pdf [Accessed 8 June 2026].
- James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021) *An introduction to statistical learning: with applications in R*. 2nd edn. New York: Springer.
- McKinsey Global Institute (2023) *The economic impact of advanced analytics in retail operations*. McKinsey & Company.
- Nand, P., Khidhir, A.M., Sharma, A., Kumar, A., Jain, S. and Tomer, V. (2025) *Forecasting retail sales using machine learning models*. ResearchGate. Available at: https://www.researchgate.net/publication/390949992_Forecasting_Retail_Sales_using_Machine_Learning_Models [Accessed 8 June 2026].
- Ravi, C. and Raman, S. (2018) A review of retail analytics and its applications in business decision making. *International Journal of Retail & Distribution Management*, 46 (7), pp. 650–675.
- Sowmya, G., Shanthi, S. and Prasad, S. (2024) *Predicting retail sales using machine learning techniques: a comparative study*. ResearchGate. Available at: https://www.researchgate.net/publication/382613584_Predicting_Retail_Sales_Using_Machine_Learning_Techniques_A_Comparative_Study [Accessed 8 June 2026].
- Venkatesh, K., Ramkumar, M. and Suresh, P. (2024) *Sales forecasting using machine learning*. ResearchGate. Available at: https://www.researchgate.net/publication/387657586_Sales_Forecasting_using_Machine_Learning [Accessed 8 June 2026].
- Xu, X., Li, Y., Wang, J. and Zhang, H. (2020) Artificial intelligence and machine learning applications in retail analytics. *Journal of Retailing and Consumer Services*, 57, p. 102240.

